SIMATS ENGINEERING

ASSESSMENT TOOL 2: SCENARIO-BASED

CLOUD COMPUTING AND BIG DATA ANALYTICS (CSA15)

Student Name	Paleru Dharani Govardhan
Register Number	192525280
Course Outcome	CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments (BL3)
Total Marks	25 (5 questions × 5 marks)

Code of Conduct

I, Paleru Dharani Govardhan (Register Number: 192525280), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Scenario-Based Questions and Answers

1. Cloud Architecture for a High-Demand Video Streaming Platform

A suitable architecture should use a global content delivery network (CDN), object storage, scalable compute services, load balancers, and monitoring. Video files should be stored in durable object storage and delivered through CDN edge locations close to viewers. Frequently watched videos are cached at the edge to reduce latency, origin-server load, and buffering. Adaptive bitrate streaming (such as HLS or MPEG-DASH) adjusts video quality to the viewer's available bandwidth.

A global DNS or traffic manager directs users to a healthy, nearby endpoint. Application load balancers distribute requests across multiple availability zones, while auto-scaling adds or removes virtual machines or containers according to CPU usage, request rate, queue length, and concurrent sessions. Stateless application services and managed databases or caches help the system scale horizontally.

For cost optimization, use CDN caching to reduce repeated origin traffic, autoscale during demand peaks, select suitable instance sizes, and use reserved capacity or savings plans for predictable baseline workloads. Store older or rarely accessed media in lower-cost storage tiers, remove unused resources, and monitor cost per stream. These measures provide high performance without paying for peak capacity all the time.

2. Container Orchestration and Multi-Cloud Deployment for a Fintech Startup

The startup should package each application as a versioned container image and store it in a secure container registry. Kubernetes can orchestrate containers across clusters, maintain the desired number of replicas, restart failed containers, perform rolling updates, and support rollback. Infrastructure and deployment configuration should be managed as code, while a common CI/CD pipeline builds, tests, scans, and deploys the same image to each cloud provider. Configuration and secrets should be managed separately from the image.

Service discovery can be handled by Kubernetes Services and cluster DNS within each cluster. For communication across clouds, use a service mesh or global traffic-management layer with authenticated service-to-service communication, health checks, and policy controls. Horizontal Pod Autoscaling can scale workloads based on CPU, memory, or custom metrics; cluster autoscaling can add worker nodes when capacity is insufficient.

Multi-cloud deployment improves resilience, reduces dependence on one provider, and may allow each provider's strengths to be used. However, it increases operational complexity, networking and data-transfer costs, security-policy differences, and the risk of inconsistent configurations. Data replication, identity federation, observability, compliance, and regular disaster-recovery tests are therefore essential. A shared deployment standard and a carefully chosen active-active or active-passive design help maintain consistency.

3. Backup and Disaster Recovery for an E-Learning Platform

The platform should combine scheduled backups, point-in-time recovery, object versioning, and replication to a separate region or account. Enable database automated backups and transaction-log retention where supported. For uploaded learning materials, enable object versioning and, where appropriate, immutable or write-protected backup copies. Keep backup credentials separate from production credentials and restrict deletion permissions to reduce the impact of accidental or malicious deletion.

Recovery Time Objective (RTO) is the maximum acceptable time to restore service; Recovery Point Objective (RPO) is the maximum acceptable amount of data loss measured in time. Frequent database snapshots and transaction-log backups reduce RPO. Warm standby infrastructure, infrastructure-as-code templates, automated restoration, and preconfigured DNS or traffic failover reduce RTO. The exact values should be set from business requirements and validated through recovery tests.

For business continuity, document recovery roles and procedures, monitor backup completion, encrypt backups in transit and at rest, and regularly test restoration using representative data. During a failure, declare the incident, isolate affected systems if necessary, restore data to a clean environment, validate data integrity and application functionality, redirect users, and communicate status. After recovery, reconcile missing transactions and review the incident to improve the plan.

4. Secure Hybrid Cloud Architecture for a Government Agency

The agency should retain regulated or highly sensitive workloads in its on-premises data centre and place approved non-sensitive services in a public cloud. A secure site-to-site VPN or dedicated private connection should link the environments. Network segmentation, firewalls, private endpoints, and strict routing rules should limit communication to approved services. Sensitive systems should not be exposed directly to the public internet, and data should be classified before placement.

Use centralized identity and access management with single sign-on, multi-factor authentication, role-based access control, and least-privilege permissions. Federated identity can allow authorized users to access both environments without maintaining unrelated accounts. Privileged access should be monitored, reviewed, and time-limited. Encrypt data in transit using TLS and data at rest using approved encryption standards. Manage keys through a controlled key-management service or hardware security module, with access logging and rotation.

Hybrid cloud challenges include inconsistent policies, network latency, complex troubleshooting, different monitoring tools, data-residency obligations, and managing identities across environments. The agency should establish common security baselines, centralized logging and alerting, configuration management, regular vulnerability assessments, compliance audits, and tested incident-response procedures. This approach balances compliance with the flexibility of cloud services.

5. Multi-Region High Availability and Fault Tolerance for a SaaS Provider

Deploy the application in at least two geographically separate cloud regions, with each region spread across multiple availability zones. A global DNS or traffic manager should direct users to healthy regions using health checks. An active-active model serves traffic from both regions and can provide faster failover; an active-passive model keeps a secondary region ready to take over and may be simpler for some stateful workloads. The choice depends on cost, consistency requirements, and recovery objectives.

Replicate data between regions using the database's supported replication method. Synchronous replication may provide stronger consistency but can add latency; asynchronous replication usually reduces latency but may lose recent writes during a sudden failure. The design must define which region is authoritative, how write conflicts are prevented, and how failover and failback are controlled. Store backups separately from live replicas because replication can also copy accidental deletion or corruption.

Use centralized monitoring for latency, error rate, traffic, CPU and memory, database replication lag, queue depth, and availability checks. Configure alerts with clear severity levels and notify the responsible team through established channels. Automated failover should be tested, while dashboards, logs, traces, and runbooks help engineers diagnose incidents. Conduct regular regional-failure drills and review RTO and RPO results to verify that the service can continue operating when one region becomes unavailable.

Conclusion

Cloud reliability depends on selecting an architecture that matches the workload, automating scaling and recovery, protecting identities and data, and continuously monitoring performance. Virtualization and container orchestration improve resource utilization and portability, while redundancy, backups, and tested failover reduce service disruption. Security controls and cost monitoring should be built into every design.